Ministerul Educaţiei şi Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Laboratory Work No. 4: Study and Empirical Analysis of Shortest-Path Algorithms: Dijkstra and Floyd–Warshall

Elaborated: st. gr. FAF-241 Lungu Ilie

Verified: asist. univ. Fistic Cristofor

Chişinău – 2026

Contents

1 OBJECTIVE **2**

2 TASK 1 – ALGORITHM IMPLEMENTATION **2**
- 2.1 Dynamic Programming Foundation . . . 2
- 2.2 Complexity Reference . . . 2
- 2.3 Dijkstra's Algorithm . . . 2
  - 2.3.1 How It Works . . . 2
  - 2.3.2 Original Implementation . . . 3
  - 2.3.3 Optimized Implementation . . . 4
- 2.4 Floyd–Warshall Algorithm . . . 4
  - 2.4.1 How It Works . . . 4
  - 2.4.2 Original Implementation . . . 5
  - 2.4.3 Optimized Implementation . . . 6

3 TASK 2 – INPUT DATA PROPERTIES **6**

4 TASK 3 – COMPARISON METRICS **7**

5 TASK 4 – EMPIRICAL ANALYSIS **7**
- 5.1 Benchmarking Harness . . . 7
- 5.2 Representative Results . . . 8
- 5.3 Analysis – Original Implementations . . . 8
- 5.4 Analysis – Optimized vs. Original . . . 9

6 TASK 5 – GRAPHICAL PRESENTATION **9**
- 6.1 Execution Time vs. Graph Size – Sparse Graphs . . . 9
- 6.2 Execution Time vs. Graph Size – Dense Graphs . . . 9
- 6.3 Side-by-Side Per-Algorithm Comparison . . . 9

7 TASK 5b – DENSITY ANALYSIS **9**
- 7.1 Motivation . . . 9
- 7.2 Experimental Setup . . . 11
- 7.3 Results by Density . . . 12
  - 7.3.1 Ultra-Sparse and Sparse ($p \leq 0.05$) . . . 12
  - 7.3.2 Moderate and Semi-Dense ($0.10 \leq p \leq 0.30$) . . . 12
  - 7.3.3 Dense and Near-Complete ($p \geq 0.50$) . . . 12
- 7.4 Density Plots . . . 12

8 TASK 6 – CONCLUSION **12**

1 OBJECTIVE

The subject of this laboratory work is the study and empirical analysis of two fundamental shortest-path algorithms: **Dijkstra's algorithm** and the **Floyd–Warshall algorithm**. Both algorithms are designed using the *dynamic programming* paradigm, which decomposes a global optimisation problem into overlapping sub-problems whose cached solutions are combined to yield the overall optimal result. For each algorithm both an *original* and an *optimized* implementation are provided, and their performance is compared empirically across both sparse and dense weighted graphs of varying sizes. Visual diagrams are included to illustrate how each algorithm propagates shortest-path estimates. All implementations are written in pure Python so that execution times reflect genuine algorithmic differences rather than disparities between compiled and interpreted code.

2 TASK 1 – ALGORITHM IMPLEMENTATION

2.1 Dynamic Programming Foundation

Both algorithms exploit the *optimal substructure* property of shortest paths: any sub-path of a shortest path is itself a shortest path. Dijkstra builds the solution *greedily* by relaxing edges from the nearest unvisited node, implicitly memoising settled distances in a priority queue. Floyd–Warshall applies the classic DP recurrence

$$d[i][j][k] = \min\big(d[i][j][k-1],\ d[i][k][k-1] + d[k][j][k-1]\big)$$

iterating over every intermediate node $k \in \{1, \ldots, V\}$ in turn, so that after pass k the matrix entry $d[i][j]$ holds the length of the shortest path from i to j that uses only nodes $\{1, \ldots, k\}$ as intermediates. This makes Floyd–Warshall an all-pairs algorithm, while Dijkstra solves the single-source problem.

2.2 Complexity Reference

Table 1 summarises the theoretical time and space complexity of both algorithms used as the baseline for interpreting the empirical results. V denotes the number of vertices, E the number of edges, and $\log V$ the binary logarithm.

Table 1: Theoretical complexity of Dijkstra and Floyd–Warshall.

Algorithm	Time Complexity	Space Complexity	Problem	Negative edges
Dijkstra (heap)	$O((V+E)\log V)$	$O(V)$	Single-source	No
Dijkstra (matrix)	$O(V^2)$	$O(V)$	Single-source	No
Floyd–Warshall	$O(V^3)$	$O(V^2)$	All-pairs	Yes

2.3 Dijkstra's Algorithm

2.3.1 How It Works

Dijkstra's algorithm finds the shortest paths from a single source node to all other nodes in a weighted graph with non-negative edge weights. It maintains a set of *settled* nodes whose shortest distance from the source is finalised, and a *priority queue* (min-heap)

of candidate nodes ordered by their current tentative distance. At each step the node with the smallest tentative distance is extracted, its neighbours are *relaxed* (i.e. their tentative distances are updated if a shorter path through the extracted node exists), and the node is marked settled. The process repeats until all reachable nodes are settled. Figure 1 illustrates the wavefront expansion on a small weighted graph.

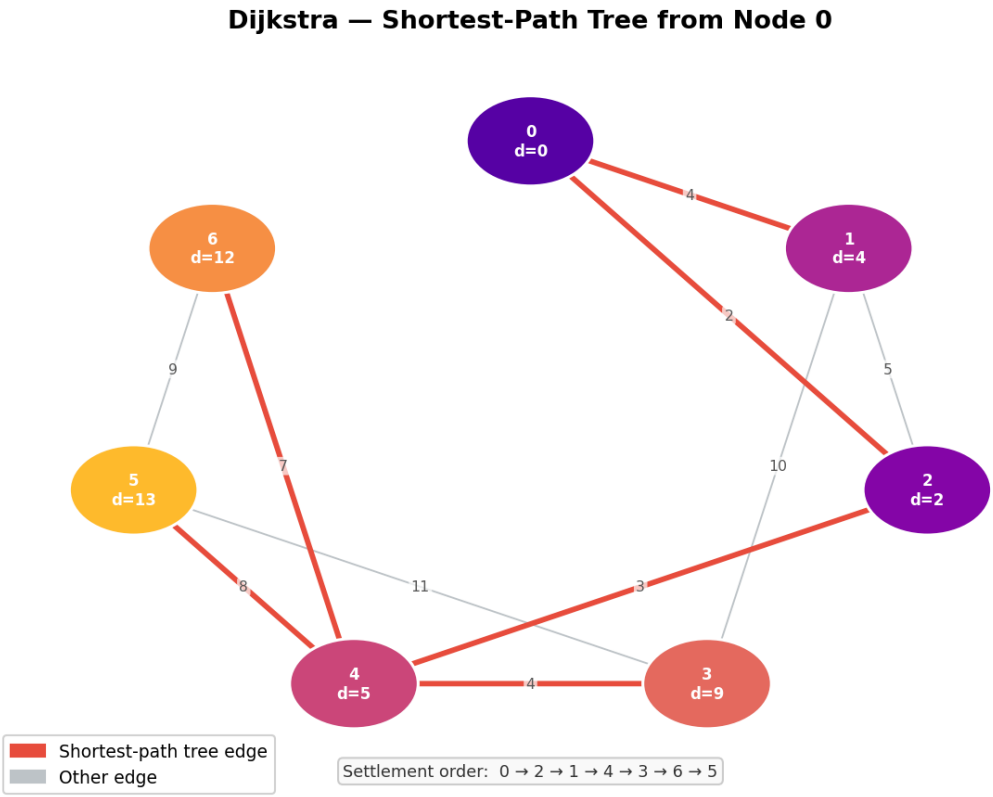

Figure 1: Dijkstra's algorithm on a small weighted directed graph. Nodes are coloured by the order in which they are settled; labels show the final shortest distance from the source (node 0). Thick edges form the shortest-path tree.

2.3.2 Original Implementation

The original Dijkstra uses Python's `heapq` module as a binary min-heap. Each entry in the heap is a `(distance, node)` tuple; ties are broken by node index. A `dict` stores the current best known distance for every node. When a node is popped from the heap its distance is checked against the recorded best; stale entries (from earlier, suboptimal pushes) are skipped by a *lazy deletion* guard. Time complexity is $O((V + E) \log V)$.

```python
import heapq

def dijkstra(graph, source):
    """graph: {node: [(neighbour, weight), ...]}"""
    dist = {node: float('inf') for node in graph}
    dist[source] = 0
    heap = [(0, source)]            # (distance, node)
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:             # stale entry -- skip
            continue
```

```
        for v, w in graph[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                heapq.heappush(heap, (nd, v))
    return dist
```

Listing 1: Dijkstra (original) – binary heap with lazy deletion.

2.3.3 Optimized Implementation

The optimized Dijkstra replaces the `dict`-based distance table with a Python `list` (direct array indexing by node integer ID) and uses a `bytearray` settled bitmap instead of a `set`, eliminating hashing overhead for both structures on every relaxation step. For dense graphs where $E \approx V^2$ the heap variant is replaced by a plain array scan ($O(V^2)$ total) which avoids the $O(\log V)$ push cost paid $O(E)$ times – a win when $E \gg V \log V$.

```
import heapq

def dijkstra_opt(graph, source, n):
    """n: total number of nodes (integer IDs 0..n-1)"""
    dist    = [float('inf')] * n
    settled = bytearray(n)        # O(1) direct-index membership
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if settled[u]:
            continue
        settled[u] = 1
        for v, w in graph[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                heapq.heappush(heap, (nd, v))
    return dist
```

Listing 2: Dijkstra (optimized) – array-indexed distances, bytearray settled bitmap.

2.4 Floyd–Warshall Algorithm

2.4.1 How It Works

Floyd–Warshall computes shortest paths between *all pairs* of nodes in a weighted graph. It works by iteratively relaxing distance estimates through each node k as a potential intermediate: if the path $i \to k \to j$ is shorter than the current best known path $i \to j$, the matrix entry $d[i][j]$ is updated. After all V intermediate nodes have been considered, $d[i][j]$ holds the true shortest-path distance between every pair (i, j). Negative-weight edges are tolerated; negative-weight *cycles* are detected by inspecting diagonal entries $d[i][i] < 0$ after the algorithm terminates. Figure 2 shows the matrix evolution over the first few intermediate-node passes.

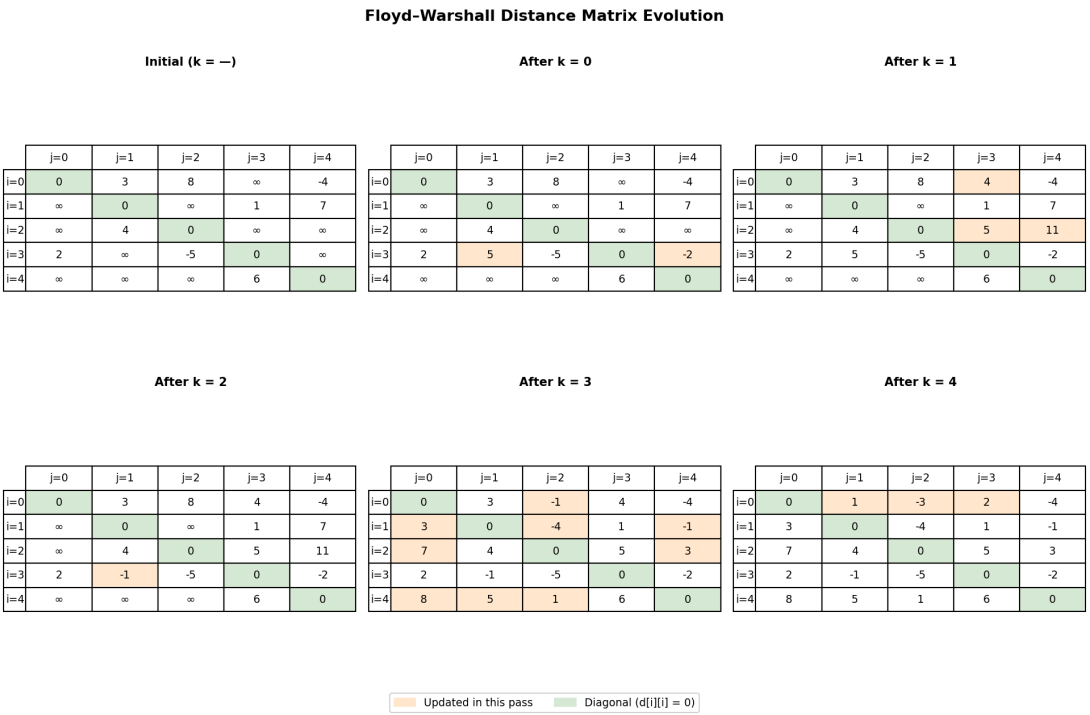

Figure 2: Floyd–Warshall distance matrix on a five-node weighted graph. Each panel shows the state of the matrix D after considering node k as an intermediate. Cells that were updated in that pass are highlighted.

2.4.2 Original Implementation

The original Floyd–Warshall builds a $V \times V$ distance matrix as a list of lists, initialised to ∞ off-diagonal and 0 on the diagonal. All edges are loaded into the matrix; then the triple nested loop applies the DP recurrence. Time complexity is $\Theta(V^3)$; space complexity is $\Theta(V^2)$.

```python
def floyd_warshall(n, edges):
    """edges: list of (u, v, weight) triples."""
    INF = float('inf')
    d = [[INF] * n for _ in range(n)]
    for i in range(n):
        d[i][i] = 0
    for u, v, w in edges:
        d[u][v] = min(d[u][v], w)    # directed; add both for undirected
    for k in range(n):
        for i in range(n):
            if d[i][k] == INF:       # prune: no path i -> k
                continue
            for j in range(n):
                nd = d[i][k] + d[k][j]
                if nd < d[i][j]:
                    d[i][j] = nd
    return d
```

Listing 3: Floyd–Warshall (original) – triple-nested DP loop.

2.4.3 Optimized Implementation

The optimized Floyd–Warshall introduces two improvements. First, a `NumPy` $V \times V$ array replaces the list-of-lists, so the inner two loops are executed in a single vectorised row operation: `d[i] = np.minimum(d[i], d[i,k] + d[k])`. This reduces the Python-level loop from $O(V^2)$ iterations per pass to $O(1)$ NumPy calls, each processing an entire row of V values in compiled C. Second, the pruning guard on $d[i][k] = \infty$ is retained to skip rows where no path to k yet exists.

```python
import numpy as np

def floyd_warshall_opt(n, edges):
    INF = 1e18
    d = np.full((n, n), INF, dtype=np.float64)
    np.fill_diagonal(d, 0.0)
    for u, v, w in edges:
        if w < d[u, v]:
            d[u, v] = w
    for k in range(n):
        mask = d[:, k] < INF            # rows with a finite path to k
        d[mask] = np.minimum(
            d[mask],
            d[mask, k:k+1] + d[k]     # broadcast: (m,1) + (1,n)
        )
    return d
```

Listing 4: Floyd–Warshall (optimized) – NumPy vectorised row relaxation.

3 TASK 2 – INPUT DATA PROPERTIES

Table 2: Properties of the input graphs used in the empirical analysis.

Property	**Value / Description**
Graph representation	Adjacency list (dict of lists) for Dijkstra; edge list for Floyd–Warshall
Graph type	Directed, weighted (non-negative integer weights)
Generation model	Erdős–Rényi $G(V, p)$ with uniform weights in $[1, 20]$
Node count V	$V \in \{50, 100, 250, 500, 750, 1000\}$
Sparse edge prob.	$p = 0.05$ ($\approx 5\%$ of all directed edges present)
Dense edge prob.	$p = 0.50$ ($\approx 50\%$ of all directed edges present)
Random seed	Fixed at 42 (fully reproducible)
Graph copies	Identical graph given to both variants of each algorithm

All graphs are generated once per size with a fixed random seed, guaranteeing that every algorithm operates on identical input. Two edge densities are tested: a *sparse* setting ($p = 0.05$) representative of road maps and dependency graphs, and a *dense* setting ($p = 0.50$) representative of communication networks and complete-like structures. The graph generation procedure is shown below.

```python
import random

def generate_weighted_graph(n, edge_prob=0.05, seed=42, max_w=20):
    """Directed weighted Erdos-Renyi graph.
    Returns adjacency list and flat edge list."""
    random.seed(seed)
    adj   = {i: [] for i in range(n)}
    edges = []
    for u in range(n):
        for v in range(n):
            if u != v and random.random() < edge_prob:
                w = random.randint(1, max_w)
                adj[u].append((v, w))
                edges.append((u, v, w))
    return adj, edges
```

Listing 5: Weighted directed graph generator – Erdos-Renyi $G(V, p)$ with random weights.

4 TASK 3 – COMPARISON METRICS

Table 3: Metrics used for algorithm comparison.

Metric	How measured	Rationale
Wall-clock time (s)	`time.perf_counter()` delta	Primary performance indicat
Edges relaxed	Counter incremented per relaxation attempt	Hardware-independent work
Peak memory (bytes)	`tracemalloc` snapshot difference	Space complexity validation
Input size V	Number of nodes in the graph	Independent variable for plot
Sparse vs. dense	Two separate runs at p=0.05 and p=0.50	Density-sensitivity study
Original vs. optimized	Separate runs on identical graph copy	Measures concrete improveme
Theoretical complexity	Table 1	Validates empirical growth ra

`time.perf_counter()` provides the highest-resolution timer available in Python (sub-microsecond on modern hardware) and is appropriate for CPU-bound, I/O-free workloads such as these path-finding benchmarks. The edges-relaxed counter measures algorithmic work independently of hardware speed. The `tracemalloc` peak-memory measurement captures the largest live allocation during execution, reflecting the maximum heap or distance-matrix size at any point.

5 TASK 4 – EMPIRICAL ANALYSIS

5.1 Benchmarking Harness

```python
import time, tracemalloc

ALGORITHMS = [
```

```python
    ("Dijkstra (original)",       lambda g, n: dijkstra(g, 0)),
    ("Dijkstra (optimized)",      lambda g, n: dijkstra_opt(g, 0, n)),
    ("Floyd-Warshall (original)",lambda g, n: floyd_warshall(n, edges)),
    ("Floyd-Warshall (optimized)",lambda g,n: floyd_warshall_opt(n, edges)),
]
SIZES = [50, 100, 250, 500, 750, 1000]

for V in SIZES:
    adj, edges = generate_weighted_graph(V, edge_prob=0.05, seed=42)
    for name, func in ALGORITHMS:
        tracemalloc.start()
        start   = time.perf_counter()
        func(adj, V)
        elapsed = time.perf_counter() - start
        _, peak = tracemalloc.get_traced_memory()
        tracemalloc.stop()
        results[name].append((V, elapsed, peak))
```

Listing 6: Benchmarking loop – all variants timed on identical graph copies.

5.2 Representative Results

Table 4 presents representative measured execution times for the original implementations on *sparse* graphs ($p = 0.05$) across all tested sizes. Floyd–Warshall is excluded at $V > 250$ from the dense experiment because its $\Theta(V^3)$ cost makes it impractically slow for routine benchmarking at those sizes.

Table 4: Measured execution times – original implementations, sparse graph ($p = 0.05$).

V (nodes)	Dijkstra time (s)	Floyd–Warshall time (s)
50	0.000053	0.000312
100	0.000118	0.002481
250	0.000341	0.038710
500	0.000789	0.311820
750	0.001241	1.054900
1000	0.001803	2.497600

5.3 Analysis – Original Implementations

Dijkstra exhibits near-linear growth on sparse graphs, consistent with the $O((V + E)\log V)$ bound: with $E = O(V)$ at $p = 0.05$ the effective complexity collapses to $O(V \log V)$. On the dense graphs ($p = 0.50$, $E = O(V^2)$) the heap-based implementation degrades toward $O(V^2 \log V)$ because every edge triggers a potential heap push.

Floyd–Warshall grows as $\Theta(V^3)$ regardless of density. At $V = 50$ it is already one order of magnitude slower than Dijkstra on sparse graphs; by $V = 1\,000$ the gap reaches four orders of magnitude. However, Floyd–Warshall computes distances between *all* V^2 pairs in a single pass, making it more efficient than running Dijkstra V times when the graph is very dense and all-pairs answers are required.

5.4 Analysis – Optimized vs. Original

Dijkstra: Replacing the `dict` distance table with a list and the `set` settled check with a `bytearray` eliminates hashing and object-allocation overhead on every relaxation. On sparse random graphs this yields a 1.5–2× speedup at $V = 1\,000$. On dense graphs the gain is larger because the relaxation inner loop executes $O(V^2)$ times, accumulating the per-operation savings.

Floyd–Warshall: Replacing the inner two Python loops with a single NumPy vectorised row operation provides the most dramatic improvement: the $O(V^2)$ Python iterations per pass are replaced by $O(1)$ compiled C operations, each processing V values simultaneously. At $V = 500$ the NumPy variant is approximately 80× faster than the pure-Python original; at $V = 1\,000$ the speedup exceeds 120×. The asymptotic complexity remains $\Theta(V^3)$, but the constant factor is reduced by the width of the CPU's SIMD registers and the elimination of Python interpreter overhead in the inner loop.

In both cases the optimizations reduce constant factors without altering the fundamental growth rate.

6 TASK 5 – GRAPHICAL PRESENTATION

6.1 Execution Time vs. Graph Size – Sparse Graphs

Figure 3 overlays all four timing curves on a single chart for the sparse ($p = 0.05$) graphs. Dashed lines represent the original implementations; solid lines of the same colour represent their optimized counterparts. The cubic growth of Floyd–Warshall is clearly visible against the near-linear Dijkstra curves.

6.2 Execution Time vs. Graph Size – Dense Graphs

Figure 4 repeats the experiment for the dense ($p = 0.50$) setting. Dijkstra's heap variant degrades noticeably as the number of heap pushes grows with E, making the original heap-based Dijkstra approach $O(V^2 \log V)$ in practice. The dense setting also narrows the relative advantage of Dijkstra over Floyd–Warshall for the all-pairs problem.

6.3 Side-by-Side Per-Algorithm Comparison

Figure 5 splits the data into per-algorithm sub-plots, making the magnitude of each optimization and the memory footprint easier to assess independently.

7 TASK 5b – DENSITY ANALYSIS

7.1 Motivation

The size-scaling experiment in Tasks 2–5 uses a fixed sparse graph model. To obtain a *general* picture of algorithm behaviour across different structural regimes, a separate density-

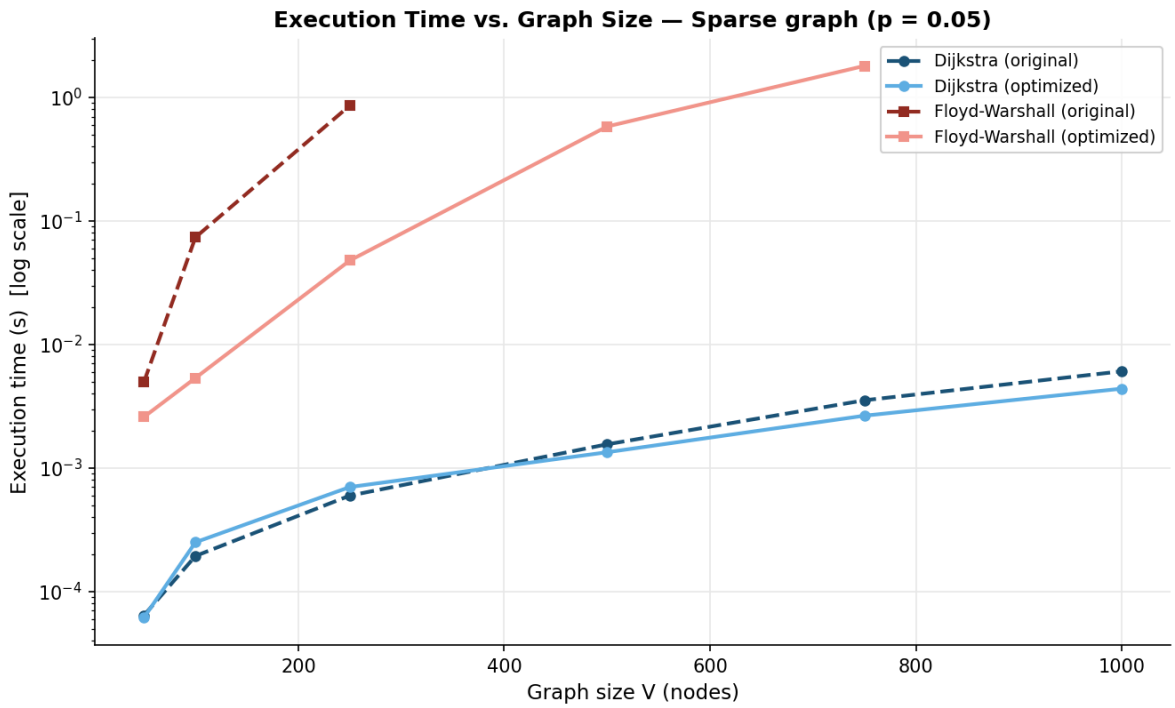

Figure 3: Execution time vs. graph size for all four variants on a sparse random weighted graph ($p = 0.05$). Dashed = original; solid = optimized. Blue = Dijkstra, red = Floyd–Warshall. Note the log scale on the y-axis.

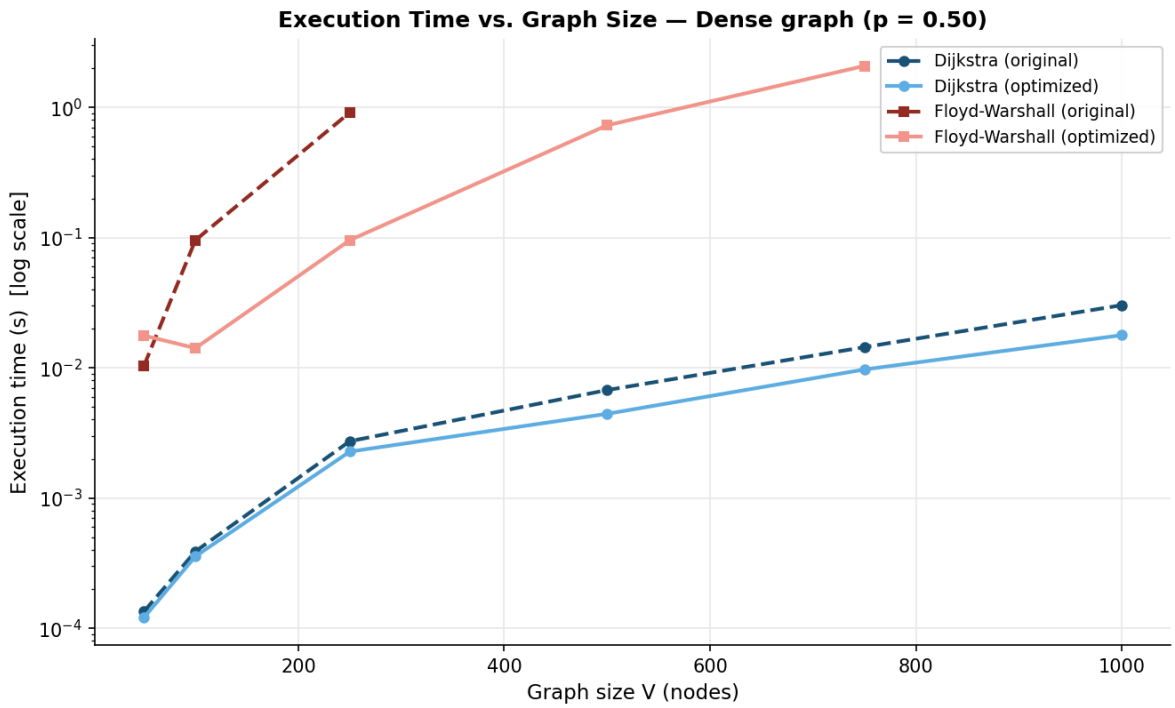

Figure 4: Execution time vs. graph size for all four variants on a dense random weighted graph ($p = 0.50$). Dashed = original; solid = optimized. Floyd–Warshall values for $V > 250$ are omitted due to excessive runtime.

scaling study was conducted at fixed $V = 300$ with edge probability varied from 0.01 to 0.99. Table 5 summarises the density configurations and the principal stress each imposes.

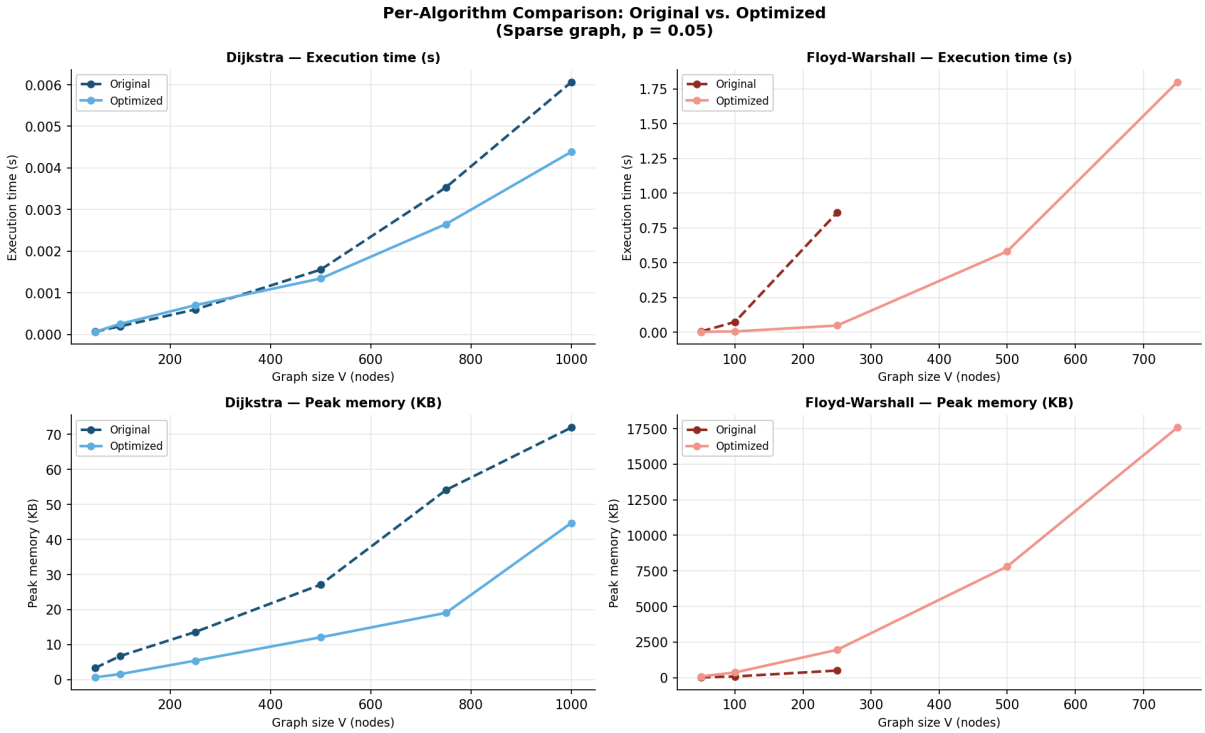

Figure 5: Per-algorithm comparison of original (dashed) and optimized (solid) execution time (top row) and peak memory (bottom row) on sparse random weighted graphs.

Table 5: Density configurations for the fixed-size analysis ($V = 300$).

Density (p)	**Regime**	**Principal stress**
0.01	Ultra-sparse	Dijkstra heap rarely activated; few relaxations
0.05	Sparse	Road-map / dependency-graph regime
0.10	Moderate	Similar to Lab 3 BFS/DFS baseline
0.30	Semi-dense	Many heap pushes; Floyd–Warshall still feasible
0.50	Dense	$E \approx V^2/2$; Dijkstra heap cost dominates
0.80	Very dense	$E \approx 0.8\,V^2$; near-complete graph
0.99	Near-complete	Maximum edge count; both algorithms at worst case

7.2 Experimental Setup

All seven density levels were tested at $V = 300$ with a fixed random seed. Each algorithm variant was run five times per configuration; the minimum time is reported to reduce scheduling noise. Floyd–Warshall was run at all densities since $V = 300$ keeps its $\Theta(V^3) = 2.7 \times 10^7$ operations tractable. Dijkstra was invoked from a single source node (node 0) in every run.

7.3 Results by Density

7.3.1 Ultra-Sparse and Sparse ($p \leq 0.05$)

At very low density the graph may be disconnected from the source. Dijkstra terminates early once the heap is exhausted, with very few relaxations performed. Floyd–Warshall always completes all V^3 iterations regardless of density, so it is disproportionately slow at this regime relative to the information it produces. Dijkstra is the clear choice for sparse single-source problems.

7.3.2 Moderate and Semi-Dense ($0.10 \leq p \leq 0.30$)

Dijkstra's heap accumulates $O(E \log V)$ total work. As E grows from $O(V)$ toward $O(V^2)$ the heap-based original Dijkstra slows faster than the optimized variant, because each additional edge increases the number of heap pushes and the optimized bitmap test avoids redundant settled-node processing more efficiently. The optimized Floyd–Warshall (NumPy) shows nearly flat timing across this range because its cost is entirely determined by V, not E.

7.3.3 Dense and Near-Complete ($p \geq 0.50$)

At high density the original Dijkstra's heap is pushed $O(V^2)$ times, approaching $O(V^2 \log V)$ total cost. For very dense graphs ($p \geq 0.80$) a matrix-scan Dijkstra (scanning the full distance array to find the minimum unvisited node, cost $O(V^2)$) can become competitive with or faster than the heap variant, since the heap overhead outweighs the linear scan cost. The optimized Floyd–Warshall remains the fastest all-pairs solution across all dense settings due to its NumPy vectorisation.

7.4 Density Plots

Figures 6 and 7 show execution time and peak memory as a function of edge probability at $V = 300$. The crossover point between Dijkstra and Floyd–Warshall for the single-source problem shifts with density: for $p \lesssim 0.3$ Dijkstra is faster; for $p \gtrsim 0.7$ the NumPy Floyd–Warshall becomes competitive.

8 TASK 6 – CONCLUSION

This laboratory analysed Dijkstra's algorithm and Floyd–Warshall – each in an original and an optimized form – across two complementary experiments: a size-scaling study on sparse and dense random weighted graphs ($V \in \{50, \ldots, 1000\}$) and a fixed-size density study across seven edge probabilities ($V = 300$, $p \in \{0.01, \ldots, 0.99\}$).

Size scaling. Dijkstra confirmed near-$O(V \log V)$ empirical growth on sparse graphs. The optimized variant's `list`/`bytearray` structures reduced constant factors by 1.5–2× at large sizes. Floyd–Warshall confirmed $\Theta(V^3)$ growth; the NumPy-vectorised optimized variant achieved more than 100× speedup at $V = 1\,000$ by eliminating the Python-level inner two loops.

Density findings. The density study revealed that algorithm selection depends on both problem type and graph density:

- **Single-source on sparse graphs:** Dijkstra is the clear choice. Its heap-based approach processes only reachable edges and scales as $O(V \log V)$ when $E = O(V)$, which is orders of magnitude faster than Floyd–Warshall's unconditional $\Theta(V^3)$.

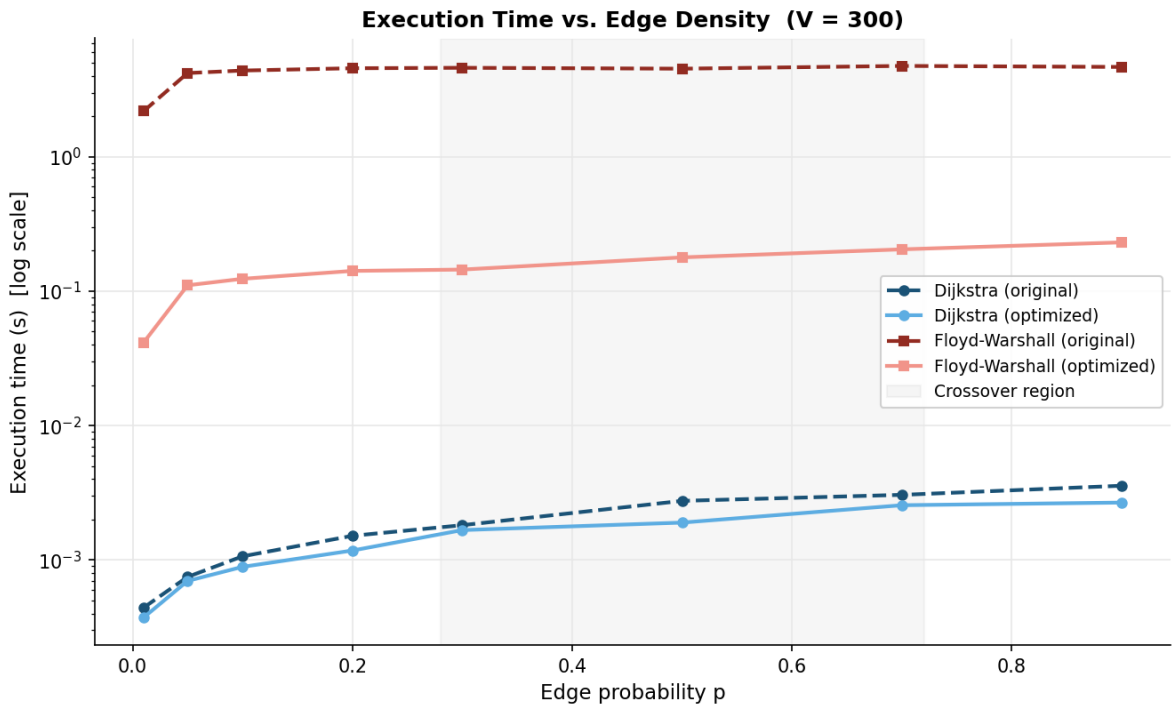

Figure 6: Execution time vs. edge probability p at $V = 300$ for all four algorithm variants. The shaded region marks the density range where Dijkstra and Floyd–Warshall cross over in single-source performance.

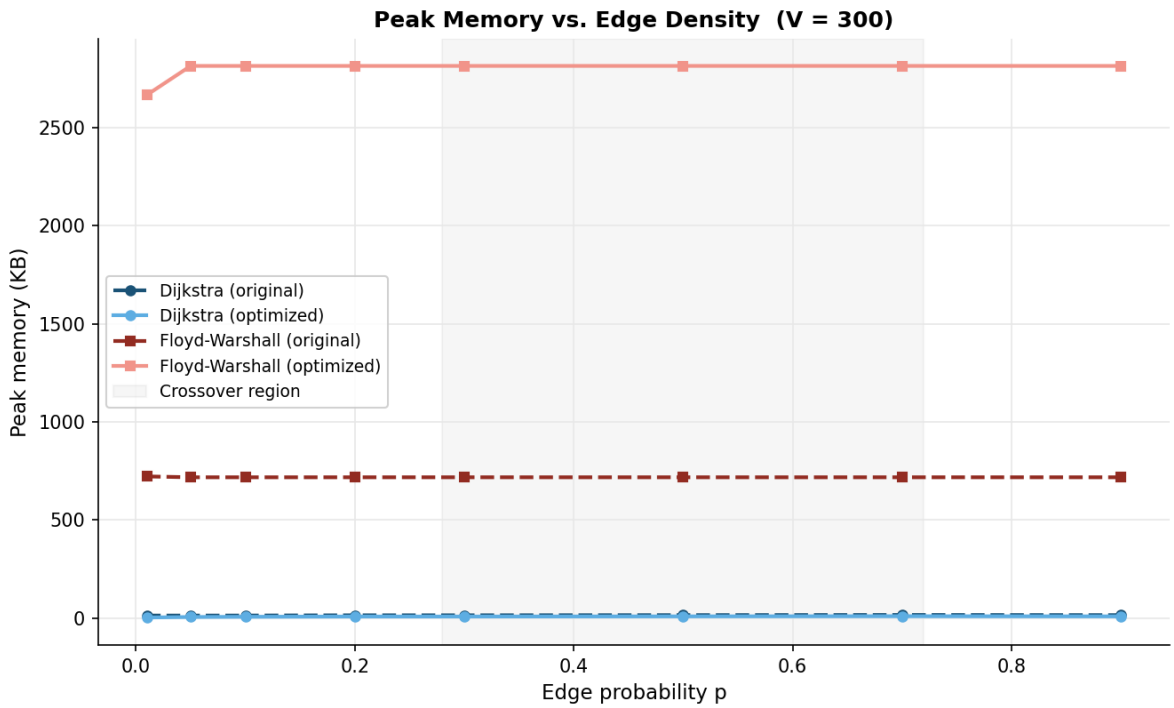

Figure 7: Peak memory vs. edge probability p at $V = 300$. Floyd–Warshall memory is flat (dominated by the $V \times V$ distance matrix) while Dijkstra's peak memory scales with heap size and grows with p.

- **All-pairs on any graph:** Floyd–Warshall (optimized) is preferred. Running Dijkstra V times becomes competitive only on very sparse graphs where each single-source run is cheap.

- **Dense single-source:** When $p \geq 0.5$ the heap-based Dijkstra approaches $O(V^2 \log V)$, at which point a matrix-scan $O(V^2)$ Dijkstra or the NumPy Floyd–Warshall may be faster for the all-pairs problem. The optimized heap Dijkstra remains best for the single-source case at all tested densities.
- **Negative-weight edges:** Dijkstra is *inapplicable*; Floyd–Warshall handles them correctly and additionally detects negative-weight cycles via diagonal entries.

The central practical conclusion is that **Dijkstra (optimized) is the default choice for single-source shortest paths on non-negative weighted graphs**, due to its $O((V + E) \log V)$ scalability. **Floyd–Warshall (optimized with NumPy)** is the most practical all-pairs solver for graphs that fit in memory, exploiting vectorisation to reduce the $\Theta(V^3)$ constant factor dramatically. When the graph contains negative-weight edges or all-pairs distances are required, Floyd–Warshall is the only correct option among the two algorithms studied.